Roadmap

- Enterprise JavaBeans - by example
  - Defining and using EJBs
  - Kinds of EJBs
  - Life-cycle
- EJB + Persistence
- EJB + Messaging
- EJB + Transactions

We shall see a sequence of simple examples that

- present the technicalities of defining and using Enterprise Java Beans,
- introduce the different kinds of EJBs - mostly focusing on the current standard, but briefly addressing the older standards as well,
- explain the inner workings of EJBs and the containers, especially the life-cycle of the components, and
- connect EJBs to technologies that were covered earlier, such as JDBC, JPA, JNDI and JMS.

We also need to understand transactions: how they are managed by the container, and how the programmer can configure or program them. Transactions should be always kept in mind when dealing with EJBs, because the applications we want to develop with EJBs are typically transactional.

Note that the examples covered on the slides are not realistic, but rather minimalistic. We shall see more meaningful examples on the labs.

What is EJB?

- Component model
- Specification
  - Focus on EJB 3
  - Current version: EJB 3.2
  - Brief introduction to EJB 2.1
- Interoperability (e.g. CORBA)

Enterprise JavaBeans is a server-side component model to build object-oriented distributed applications. EJB is not to be confused with the JavaBeans component model.

EJB is a specification, which must be implemented by Java EE application servers. As of writing, the newest version of the specification is EJB 3.2, obligatory part of Java EE

7. This course will focus on EJB 3, addressing novelties of EJB 3.1 and EJB 3.2 where necessary, but some basic information on EJB 2.1 will also be provided.

One of the key issues is interoperability. Different implementations of the specification are able to execute the same EJB component. The implementations can interoperate as well. Much of EJB is based on CORBA, and this allows interoperability with CORBA components written either in Java or in other languages.

EJB Characteristics

- Contains business logic
- Managed by the container
- Client access is mediated by the container
- EJB can be assembled without the source code
- Customized at deploy time by editing its environment
- Service information (e.g. transactions, security attributes) can be provided
- EJB can be deployed to any compliant container

EJB is for describing business logic in the business tier of a multi-tier distributed application. EJB components are executed in EJB containers that provide several services, such as life-cycle, multi-threading, transactions, security, clustering and so on.

EJB components is not to be accessed directly by other components. The clients of an EJB use the component through the container. Therefore the container can put its services into effect.

The services can be configured, though typically default behaviour is also specified. Configuration can be provided in the source code, i.e. before compilation. However, EJB components can be ``assembled'' (this is a verb with special meaning in EJB) without the source code. In that case configuration is done after compilation. Configuration can even be postponed until deployment time. Some necessary properties are not known until then, e.g. location of used services, such as IP address of a database server.

The same EJB can be deployed in different containers. Some of the configurable properties are standardized, but there are some others which are not. These latter properties can be provided in a container-specific way.

Why EJB?

- Distributed (e.g. client-server) applications?
  - Sockets?
  - RMI, RMI-IIOP, CORBA?
  - JMS?

- Web applications?
- Containers such as Spring Framework?

Purpose of EJB technology

Easy development of complex systems from scalable components

Why do we need EJBs? Why don't we simply use the technologies learnt earlier, such as programming distributed applications with sockets, or if it's not sufficiently convenient, with RPC-based solutions? Or with messaging, if loosely coupled components are preferred? Because EJBs can rely on lots of useful services provided by the EJB container, and this makes development much easier.

Moreover, we do not want to restrain ourselves and program web applications. The more complex the applications are, the more benefits the business tier provides. EJBs foster right structuring of programs, namely the strong separation of business logic and presentation logic by formally declaring the use of a separate business tier.

And why don't we use Spring Framework? The key reason here is scalability. EJB containers provide distributed server side seamlessly, since EJBs can run in several container instances, e.g. distributed in a cluster.

Kinds of Enterprise JavaBeans

- Session bean
  - Stateless
  - Stateful
  - Singleton
- Message-driven bean
- Entity bean (only in EJB 2.1)

We will learn about session beans first, in particular stateless session beans. These are perhaps the most simple of all. There are stateful session beans as well, this will be covered next. Then singleton session beans will be explained. Message-driven beans are based on message-oriented middleware and Java Message Service. Finally, we should briefly address EJB 2.1, and within this topic we will also meet entity beans. Entity beans are made obsolete by JPA entities.

Stateless session bean

What is a session? Why stateless?

- Offers some service to clients
- Acts on behalf of the client
- Client calls methods
- No *conversational state*
- Method calls are independent

The first example

Class diagram

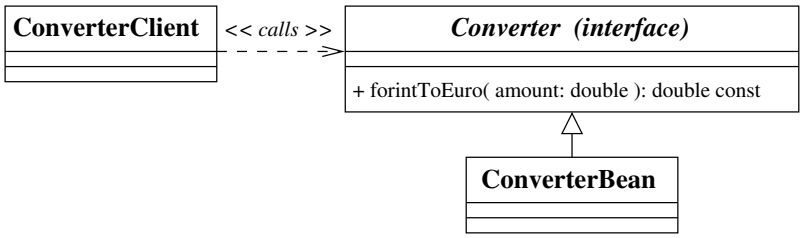


Figure 1: Example from Sun's Java EE Tutorial (simplified)

We take our first example from the Java EE Tutorial published by Sun Microsystems. We are about to see the first version of this very first example, and then investigate different variations of the theme.

The interface of the service

```
public interface Converter {
  double forintToEuro( double amount );
}
```

- A POJO interface

First of all, consider this interface. It is often referred to as a POJO, or Plain Old Java Object, because there is no sign of using any advanced technologies here. You can compile, use, or even unit test POJOs easily, without relying on sophisticated runtime environments, containers etc. However, what we see here is an interface similar to the one used in RMI. It lists the ``remote methods'', the operations that are accessible to other components in the system. These other components can be other EJBs or web components or application client components... We'll see more of these soon.

One important thing to observe: the `java.rmi.RemoteException` is missing from the `throws`-clause. It is not used explicitly in EJB. However, other rules of RMI, like the use of serializable parameters, still apply!

The implementation of the service

```
public interface Converter {
  double forintToEuro( double amount );
}

@javax.ejb.Stateless
@javax.ejb.Remote
public class ConverterBean implements Converter {
  public static final double rate = 253.2;
  public double forintToEuro( double amount ){
      return amount / rate;
  }
}
```

Now we can see how a remote interface can be implemented with a class. What we see here is not the best possible solution. It has advantages as well as disadvantages. We'll consider alternatives soon.

We regard this class a POJO, but with two EJB-specific annotations. The `Stateless` annotation refers to a special kind (the simplest kind) of EJB components, namely *stateless session beans*. The `Remote` annotation specifies that this component is accessible to components running in a different virtual machine. This is how we know that the bean is accessed with some RPC-based communication, in particular with RMI over IIOP.

Remote access of bean

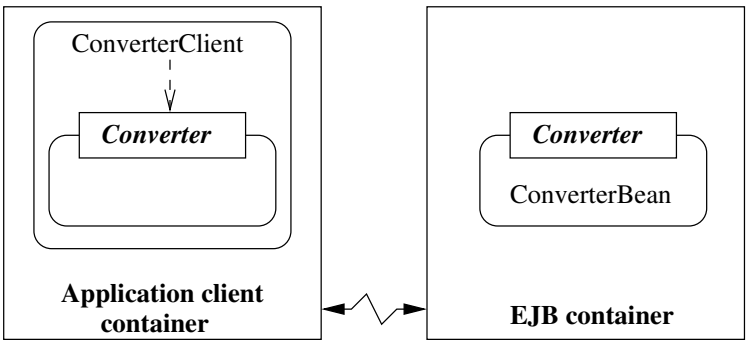


Figure 2: Containers connected by a network

We can design a client for the `Converter` EJB component. This client presented here is a special kind of component, namely an *application client component*, which can run in an *application client container*. For instance, *GlassFish* supports this kind of components - JBoss, however, does not.

The components communicate with each other through the containers. The method call from the client to the bean is carried out through a proxy object, which implements the same business interface as the bean.

The client of the service

```
public interface Converter {
  double forintToEuro( double amount );
}
```

```
import javax.ejb.EJB;
public class ConverterClient {
   @EJB(mappedName="Converter") public static Converter converter;
   public static void main( String[] args ){
       System.out.println( converter.forintToEuro(524.8) );
   }
}
```

- Dependency Injection (DI)

Our `ConverterClient` accesses the converter bean using the `EJB` annotation. Remember that this code will be executed in a container, and this container will be responsible to provide a value to the static `converter` field of `ConverterClient`. If you try to execute this code as it is, e.g.

```
java ConverterClient
```

then a `NullPointerException` will be thrown, since the field is initialized to `null`. However, if you start an appropriate container, and let this container initialize and call your component (Inversion of Control), then the container will initialize this field using *dependency injection*. The container will realize that this (application client) component depends on another component, a bean with `Converter` interface, and attempts to resolve this dependency. The resolution may need some additional information: in this case we have provided a name that can be used to find a named entity in a distributed application.

The `mappedName` annotation element specifies the name to which the bean is bound in the used naming service. Why do we use `Converter` here as the `mappedName`? This seems logical, but a bit ad-hoc here. It turns out that this name works fine with GlassFish. However, the EJB specification clearly explains that this is not portable.

``The `mappedName` element is a product-specific name that the bean reference should be mapped to. Applications that use mapped names may not be portable.''

So this will do for a first, minimalistic application, but we need something better.

Java source files

- Converter.java
- ConverterBean.java
- ConverterClient.java

Compilation

```
javac -classpath .:javaee.jar *.java
```

Pointless to run like this:

```
java -classpath .:javaee.jar ConverterClient
```

- `NullPointerException`

So far we have written three type definitions, organized in three source files. These can be compiled, once we resolve the dependency for the package `javax.ejb`. This package is not part of Java SE, we need the API for Java EE. Assuming that we have a `javaee.jar` file containing the three referenced annotation types (`Stateless`, `Remote` and `EJB`), we can compile the Java sources with this jar in the classpath.

If you have a GlassFish, you can find `glassfish/lib/javaee.jar` in its installation directory. (Obviously, on Windows use backslash instead of slash, and use semicolon instead of colon in path expressions, e.g. in `classpath`.)

Assembling the components

- EJB component
- Application client component

In order to use the bytecode produced by the compiler, we need to assemble two components, and run them in the appropriate containers. In what follows, we assume that a Java EE implementation is already installed on our machine. For instance, go to `https://glassfish.java.net` and download the GlassFish server.

We are about to assemble two components: an EJB component (from `Converter` and `ConverterBean`) and an application client component (from `Converter` and `ConverterClient`). These can be developed and assembled independently, once the interface (`Converter`) is specified.

Assembling the EJB component

- Components are assembled into jar files
- Contains bytecode, configuration files, resource files
- The suffix can be `.jar`, `.war` and `.ear`
- For EJB components `.jar` is used

Create the jar file

```
jar cvf ConverterEJB.jar Converter.class ConverterBean.class
```

Components are easy to pass around as Java archives: the class files together with configuration information and any further needed resource files (e.g. HTML files in case of a web-component) are collected in a jar file. Depending on the kind of the component, the suffix of this jar file can be `.jar` (this is the case in this example), or `.war` (for *web archives*), or `.ear` (for *enterprise archives*, see later).

We can create the jar file using the above `jar` command. In this very simple case no extra configuration is needed - the default configuration will do.

Assembling the application client component

- Executable jar file
- Contains a *manifest* file

Manifest file `conv.mf`

```
Manifest-Version: 1.0
Main-Class: ConverterClient
```

Create the jar file

```
jar cvfm Client.jar conv.mf Converter.class ConverterClient.class
```

Similarly to EJB components, an application client component is also assembled into a jar file, but now containing a manifest file as well. The main class, that is the class containing the `main` entry point (a `public static void main(String[])` method), of the jar file is specified in this manifest file. This turns the jar file into an executable jar file. But we are not going to run it as an executable jar file, i.e. as `java -jar Client.jar`, because this would obviously still cause a `NullPointerException`.

Deploying the EJB component

- Start a container
- For example the GlassFish application server
- Deploy the component
  - Using the administrative interface
  - Making use of the autodeploy feature

Start GlassFish

glassfish-dir`/bin/asadmin start-domain domain1`

Autodeploy

`mv ConverterEJB.jar` *glassfish-dir*`/glassfish/domains/domain1/autodeploy`

Now let's put the EJB component into effect. We have to deploy it into an EJB container. First let's start one, for instance as part of GlassFish. We can start GlassFish as written above, assuming that *glassfish-dir* is the installation directory of our GlassFish server.

Once the server is up and running, we can use its administrative interface to deploy components, but we can also make use of its autodeploy feature, and simply copy the jar file containing the component into an appropriate directory. Autodeploy is very useful during development, when the container is on the local host. Of course we prefer the administrative interface for production deployment, for instance when we have to access an application server on a different machine.

Deploy and run the application client component

- Start an application client container
- Make it execute the application client component
- First we do it on the same machine

Start container and client

glassfish-dir`/glassfish/bin/appclient -jar Client.jar`

Finally we have to run the application client component in an appropriate container. GlassFish has such a container. For the sake of simplicity, we assume that the same machine is used for running the application server and the application client container.

Configure for realistic execution

- Need to specify target server
- Need to authenticate
- More technical details on labs

The execution was rather simplistic so far. To prepare for more realistic execution, further information is needed. For instance, you need to specify server name (and port) for distributed execution, and you need to authenticate if the server is configured with authentication enabled.

Some variations

Portable names

```
import javax.ejb.EJB;

public class ConverterClient {

   @EJB(lookup="java:global/ConverterEJB/ConverterBean!Converter")
   public static Converter converter;

   public static void main( String[] args ){
      System.out.println( converter.forintToEuro(524.8) );
   }

}
```

We have learnt that `mappedName` is not portable. Many portable solutions exist, one is the use of *session bean portable global JNDI name*. Note the `java:global` namespace here, and remember that the bean was assembled in `ConverterEJB.jar`, the name of the bean class was `ConverterBean`, and we are using it through its remote business interface `Converter` here.

We will learn other solutions soon.

Configuration of components

- Annotations
  - Within Java
  - Less files, more tangled code
  - Default values

- Deployment descriptors
  - XML configuration
  - Annoying to read and write
  - Real POJOs
  - Can be more precise

Example

- Modify application client component
- Introduce deployment descriptor

There are two ways to provide configuration information for a component. The one shown in the first example was by using annotations, the other one is by using *deployment descriptors* - which are XML files with predefined structure. Many developers prefer annotations, because they are more compact: no need to read and write (overly verbose) XML, and defaulting mechanism is also provided. Altogether, there are less files and less ELOC with annotations. However, the code becomes more tangled, since component configuration is not modularized. Deployment descriptors allow us to write real POJOs, by factoring out component configuration into a description written in a ``declarative domain specific language''. Furthermore, deployment descriptors offer slightly more configuration possibilities than annotations.

Deployment descriptor

- Stored in `META-INF` directory
- Standardized deployment descriptor
  - Contains information that is already standardized
  - E.g. `application-client.xml`
  - Structure: `http://java.sun.com/xml/ns/javaee/application-client_5.xsd`
- Product-specific deployment descriptor
  - Often also an XML document
  - E.g. `sun-application-client.xml`
  - Structure: `http://www.sun.com/software/appserver/dtds/sun-application-client_5_0-0.dtd`

Configuration information can be described in the *deployment descriptor* of the component. Some information can be described in a standardized manner: we can use a portable XML document for this purpose. The structure of this document is specified by the EJB specification. For instance, application client components can be configured in a file named `META-INF/application-client.xml`, and its structure is specified in the above XML schema.

Information that is not (yet) standardized can be provided in a product specific deployment descriptor. In many implementations of the Java EE standard, including GlassFish, product-specific deployment descriptors are also XML files. Application client components in GlassFish can make use of a `META-INF/sun-application-client.xml` file. The structure of this file is specified in a DTD.

Pure POJO application client component

```
public class ConverterClient {

   public static Converter converter;

   public static void main( String[] args ){
       System.out.println( converter.forintToEuro(524.8) );
   }

}
```

We can completely get rid of any static dependencies on Java EE API in the Java source code. You can compile this without the `javaee.jar` in the classpath. Note, however, that you cannot run this code without the container filling out the `converter` field. So, for example, testing this code is still not straightforward.

application-client.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<application-client xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  metadata-complete="true" version="5"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/application-client_5.xsd">
 <display-name>converter-app-client</display-name>
 <ejb-ref>
  <ejb-ref-name>ConverterEJB</ejb-ref-name>
  <ejb-ref-type>Session</ejb-ref-type>
  <remote>Converter</remote>
  <injection-target>
   <injection-target-class>ConverterClient</injection-target-class>
   <injection-target-name>converter</injection-target-name>
  </injection-target>
 </ejb-ref>
</application-client>
```

We can provide configuration details on dependency injection requirements imposed on the container using the deployment descriptor. In this solution an `EJB reference` is provided. The reference `ConverterEJB` still needs to be bound to a bean, which must be a session bean with a remote interface of the type `Converter`, injected into the field `ConverterClient.converter`.

The header of the above XML document refers to the schema specifying the structure of the document. Note that you should not break the line within the string - I just wanted the content to fit on the page.

sun-application-client.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE sun-application-client PUBLIC
  "-//Sun Microsystems, Inc.//DTD Application Server 9.0
     Application Client 5.0//EN"
  "http://www.sun.com/software/appserver/dtds/
     sun-application-client_5_0-0.dtd">
<sun-application-client>
 <ejb-ref>
  <ejb-ref-name>ConverterEJB</ejb-ref-name>
  <jndi-name>Converter</jndi-name>
 </ejb-ref>
</sun-application-client>
```

Here we have the product-specific part of the configuration, in the other XML document. Again, the header is broken here into several lines for legibility. You can observe the reference to a DTD specifying the structure of the XML document.

The EJB reference `ConverterEJB` is bound to a bean with a JNDI name `Converter`. We already know that this JNDI name is specific to GlassFish. Can you modify this XML so that it uses a portable JNDI name?

Assemble the component

- Directory structure respects packages
- Deployment descriptors in `META-INF`

```
jar cvfm Client.jar \
    conv.mf Converter.class ConverterClient.class META-INF
```

When we assemble the application client component with the deployment descriptors, we have to preserve the directory structure in the jar file. The class files go to the root; if the definitions are in some packages, the directory structure follows the package hierarchy. The deployment descriptors go into the `META-INF` directory.

Standalone application client

- What if no application client container is available?
- Especially some older Java EE implementations
- JBoss AS 6
- No dependency injection without container
- Good old JNDI lookup

There are some application servers, especially older ones, like the widely used JBoss AS 6, that do not support application client components: they simply do not have an application client container. Without a container, a client cannot make use of certain services, for instance dependency injection. In such a standalone application client, we can find a communication partner, e.g. an EJB component, by looking it up in a JNDI naming service. In earlier versions of Java EE, which did not support dependency injection, all lookups were performed in JNDI, even in containers. Dependency injection makes life easier, but in certain situation, like in the case of standalone application clients, the good old JNDI lookup is the right solution.

Client code for JBoss AS 6

```
import javax.naming.*;

public class ConverterClient {

   static final String name = "ConverterBean/remote";

   public static void main (String[] args) throws NamingException {
      Context ctx = new InitialContext());
      try {
         Converter converter = (Converter)ctx.lookup(name);
         System.out.println( converter.forintToEuro(524.8) );
      } finally { ctx.xlose(); }
   }

}
```

If we prepare a standalone application client for JBoss AS 6, we can use JNDI to look up the converter bean. The JNDI name `ConverterBean/remote` is specific to JBoss, it's not portable. (By the way, the Java EE specification version to which JBoss AS 6 was developed, had made no requirements on portable JNDI names yet.)

Note that `javax.naming.Context`, for some reason, is not a subtype of `java.lang.AutoCloseable`, and therefore it cannot be used in a *try-with-resources* statement. This is why an old-fashioned *try-finally* construct is used here. By the way, `Context` is expected to be `AutoCloseable` in Java 8.

Running the client

- Set up JNDI (e.g. put a `jndi.properties` file in the classpath)
- Run with JBoss client libraries

jndi.properties

```
java.naming.factory.initial=org.jnp.interfaces.NamingContextFactory
java.naming.provider.url=jnp://localhost:1099/
java.naming.factory.url.pkgs=org.jboss.naming:org.jnp.interfaces
```

Compile and run

```
javac ConverterClient.java
java -classpath .:jbossall-client.jar ConverterClient
```

In order to access the naming service, JNDI has to be configured, e.g. with a `jndi.properties` file that is placed in the classpath. In this client there is no static dependency on the Java EE API, it can be compiled with Java SE. For running it, however, we need the application server specific libraries, which are packed into the `jbossall-client.jar` file in the case of JBoss AS 6.

Annotating the business interface

- In 1st example the business interface was not annotated
- The bean class was annotated with `@Remote`
- The interface can also be annotated

```
@javax.ejb.Remote public interface Converter {
  double forintToEuro( double amount );
}

@javax.ejb.Stateless
public class ConverterBean implements Converter {
  public static final double rate = 253.2;
  public double forintToEuro( double amount ){
      return amount / rate;
  }
}
```

The effect of these type definitions is the same as that of those in the first example.

Assembling an enterprise archive

- A component is assembled in a jar file
- All the components of an application can be assembled into ear file
- This is a jar file with special suffix
- It can be deployed

```
jar cvf ConverterEJB.jar ConverterBean.class Converter.class
jar cvfm Client.jar conv.mf ConverterClient.class Converter.class
jar cvf ConverterApp.ear ConverterEJB.jar Client.jar
```

Sometimes we want to assemble all the components of an application into a single file, called an *enterprise archive*. This is a jar file with the `.ear` suffix. Once we have the jar files containing the components, we can pack them, together with application configuration files, into the enterprise archive. Such an enterprise archive can be deployed into the application server as well. With GlassFish, you can use either the administrative interface, or the autodeploy feature.

The EJB annotation

According to the API reference:

`String beanName`

The ejb-name of the Enterprise Java Bean to which this reference is mapped.

`String lookup`

A portable lookup string containing the JNDI name for the target EJB component.

`String mappedName`

The product specific name of the EJB component to which this ejb reference should be mapped.

`String name`

The logical name of the ejb reference within the declaring component's (e.g., `java:comp/env`) environment.

The `javax.ejb.EJB` annotation provides a number of possibilities to find a referenced been in a component. Some relevant annotation elements are enumerated here, with definitions copied from the API reference.

String beanName

The ejb-name of the Enterprise Java Bean to which this reference is mapped. Only applicable if the target EJB is defined within the same application or stand-alone module as the declaring component.

String lookup

A portable lookup string containing the JNDI name for the target EJB component.

String mappedName

The product specific name of the EJB component to which this ejb reference should be mapped. This mapped name is often a global JNDI name, but may be a name of any form. Application servers are not required to support any particular form or type of mapped name, nor the ability to use mapped names. The mapped name is product-dependent and often installation-dependent. No use of a mapped name is portable.

String name

The logical name of the ejb reference within the declaring component's (e.g., `java:comp/env`*) environment.*

Examples on the use of `@EJB`

ConverterBean

```
@Stateless(name="aName",mappedName="aMappedName")
```

ConverterClient

```
@EJB(mappedName="aMappedName")
@EJB(name="reference1")
```

```
@EJB(beanName="aName")
@EJB(name="reference2")
```

application-client.xml

```
<ejb-ref>
   <ejb-ref-name>reference1</ejb-ref-name>
   <remote>Converter</remote>
   <ejb-link>ConverterEJB.jar#aName</ejb-link>
</ejb-ref><ejb-ref>
   <ejb-ref-name>reference2</ejb-ref-name>
   <mapped-name>aMappedName</mapped-name>
</ejb-ref>
```

Some of the possibilities to link the bean to its client are illustrated here. You can provide annotation elements to the `Stateless` annotation as well. For example, a `name` in the `Stateless` annotation is matched with the `beanName` annotation element in the `EJB` annotation. The use of `mappedName` is not portable, it is better to avoid it. Introducing an additional level of indirection is sometimes a good approach. Here an EJB reference is created using the deployment descriptor. The client component uses the reference name with the `name` annotation element of the `EJB` annotation, and the EJB reference is resolved in the deployment descriptor to the bean either with the `ejb-link` tag or with the `mapped-name` tag.

The `ejb-link` tag is applicable only when the components are assembled together: if two beans are put into the same ejb jar, or two component jars are put into an ear.

Using only deployment descriptors

- Let none of the type definitions contain annotations
- Configure EJB in deployment descriptor
- Let us write an application client component
- Let us avoid dependency injection
- Configure client in deployment descriptor

In the following example configuration information will be given in deployment descriptors, both at the bean's side and at its client's side. Neither annotations, nor dependency injection will be used here. Lookup will proceed ``in a good old way'', with explicit JNDI call.

No annotations in bean

```
public interface Converter {
  double forintToEuro( double amount );
}

public class ConverterBean implements Converter {
  public static final double rate = 253.2;
  public double forintToEuro( double amount ){
      return amount / rate;
  }
}
```

Application client component without DI

- The JNDI context `java:comp/env` is available in all components
- The container configures it

- EJB references are typically put under `ejb`
- Let the EJB reference name be `myBean` here

```
import javax.naming.*;
public class ConverterClient {
   public static void main (String[] args) throws NamingException {
      Context ctx = new InitialContext();
      try {
           Converter converter =
                 (Converter)ctx.lookup("java:comp/env/ejb/myBean");
           System.out.println( converter.forintToEuro(524.8) );
      } finally { ctx.close(); }
   }
}
```

Depoyment descriptors

Fragment from `ejb-jar.xml`

```
<session>
  <ejb-name>conv</ejb-name>
  <business-remote>Converter</business-remote>
  <ejb-class>ConverterBean</ejb-class>
  <session-type>Stateless</session-type>
</session>
```

Fragment form `application-client.xml`

```
<ejb-ref>
  <ejb-ref-name>ejb/myBean</ejb-ref-name>
  <remote>Converter</remote>
  <ejb-link>ConverterEJB.jar#conv</ejb-link>
</ejb-ref>
```

The two deployment descriptors connect the bean to its client. Note that the two components are expected to be in an enterprise archive, because `ejb-link` is used here.

Depoyment descriptors: another solution

Fragment from `ejb-jar.xml`

```
<session>
  <ejb-name>conv</ejb-name>
  <mapped-name>NationalBank</mapped-name>
  <business-remote>Converter</business-remote>
  <ejb-class>ConverterBean</ejb-class>
  <session-type>Stateless</session-type>
</session>
```

Fragment form `application-client.xml`

```
<ejb-ref>
  <ejb-ref-name>ejb/myBean</ejb-ref-name>
  <ejb-ref-type>Session</ejb-ref-type>
  <remote>Converter</remote>
  <mapped-name>NationalBank</mapped-name>
</ejb-ref>
```

In this case not the `ejb-link` tag is used, but `mapped-name`. Both descriptors contain a `mapped-name`. Moreover, the EJB reference in the client's descriptor has to describe the type of the bean, namely `Session`. This solution is known to be not portable, but it allows us to deploy these components independently, without assembling them into an enterprise archive.

Depoyment descriptors: yet another solution

Fragment from `ejb-jar.xml`

```
<ejb-name>conv</ejb-name>
```

Fragment from `application-client.xml`

```
<ejb-ref-name>ejb/myBean</ejb-ref-name>
```

Fragment from `sun-ejb.xml`

```
<sun-ejb-jar><enterprise-beans><ejb>    <ejb-name>conv</ejb-name>
   <jndi-name>java:global/ConverterEJB/conv!Converter</jndi-name>
</ejb></enterprise-beans></sun-ejb-jar>
```

Fragment form `sun-application-client.xml`

```
<sun-application-client><ejb-ref>
   <ejb-ref-name>ejb/myBean</ejb-ref-name>
   <jndi-name>java:global/ConverterEJB/conv!Converter</jndi-name>
</ejb-ref></sun-application-client>
```

Instead of the `mapped-name` tag, one can use product specific deployment descriptors that let the JNDI names be matched. Portable JNDI names are used in this example. We have seen such names before, and now it's time to talk about their syntax. If the bean is deployed independently, that is in an *ejb-jar* as opposed to an enterprise archive, the syntax of the portable JNDI name in the global name-space is as follows.

```
java:global/<module-name>/<bean-name>[!<fully-qualified-interface-name>]
```

Explained in the EJB specification:

> `<module-name>` is the name of the module in which the session bean is packaged. In a stand-alone ejb-jar file or `.war` file, `<module-name>` defaults to the base name of the module with any filename extension removed. […] The default module name can be overridden using the `module-name` element of `ejb-jar.xml` file (for ejb-jar files) or `web.xml` file (for `.war` files).
>
> `<bean-name>` is the `ejb-name` of the enterprise bean. For enterprise beans defined via annotations, it defaults to the unqualified name of the session bean class, unless otherwise specified by the `name` element of the Stateless […] annotation. For enterprise beans defined via the `ejb-jar.xml` file, it is specified in the `ejb-name` deployment descriptor element.

Why is `<fully-qualified-interface-name>` optional? Later on we shall see examples on defining EJBs without a business interface.

EJB as a client of another EJB

- Three components
  - `Converter` bean
  - `Main` bean
  - `Caller` application client
- `Converter` as before
- `Main` accesses `Converter`

- `Caller` accesses `Main`

The interface of the `Main` bean

```
public interface Main {
   Object main( String[] args );
}
```

In this example we can find two EJB components, one is the client of the other. This is no more complicated than the previous examples. The `Main` bean will get access to the `Converter` bean via dependency injection. The business interface of the `Main` bean is a POJO.

DI in an EJB

```
import javax.ejb.*;
@Stateless(name="MainEJB") @Remote(Main.class)
public class MainBean implements Main {
   @EJB(name="ejb/converter") Converter converter;
   public Object main( String[] args ){
      try {
         return converter.forintToEuro(Integer.parseInt(args[0]));
      } catch( Throwable t ){ return t; }
   }
}
```

This bean is a remotely accessible, stateless session bean. Note how the class object of the business interface is passed to the `Remote` annotation: this trick is especially useful when the bean class implements more than one interface, and the `Remote` annotation, for some reason, is attached to the bean class rather than to the interface declaration.

Another important detail is that the `EJB` annotation is attached to an instance field. In an application client component, this annotation is attached to a static field. The reason is obvious. An application client component is a kind of *procedural* component, with the `main` method as the entry point, and its class is not instantiated by the container. On the contrary, the bean is more *object-oriented*, it will be instantiated (typically more than one instance will be created) by the container.

Not only (either static or instance) fields can be annotated with `EJB`. One can write *setter* methods that have a parameter typed with the appropriate business interface, and annotate this method with `@EJB`.

```
@EJB void setConverter( Converter c ){ ... }
```

Dependency injection will work here by invoking this method with the to-be-injected reference as actual parameter. The programmer can write code in this method to save the reference at his convenience. Testing components can be facilitated using this approach, since setting mock objects might become easier.

Finally, we briefly mention here that the bean class can also be annotated with `@EJB`. In this case, however, dependency injection will not occur; the bean has to look up the other bean in JNDI explicitly.

Environment entries

- Configuration of components
- Setting at deployment, e.g. in deployment descriptor
- Available in `java:comp/env`
- Dependency injection is possible
  - `@Resource` or `<injection-target>`
  - Deployment descriptor may override annotation
- An EJB reference (`@EJB`) is a special environment entry
- *Simple* environment entries
  - Wrappers of primitive types
  - `String`
  - `Class`
  - enums

Dependency injection with annotation

```
import javax.ejb.*;
import javax.annotation.Resource;

@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {

    @Resource private double rate;

    public double forintToEuro( double amount ){
        return amount / rate;
    }

}
```

Value in deployment descriptor

```
<?xml version="1.0" encoding="UTF-8"?>
<ejb-jar [...]>
<enterprise-beans>
    <session>
        <ejb-name>ConverterEJB</ejb-name>
        <env-entry>
            <env-entry-name>ConverterBean/rate</env-entry-name>
            <env-entry-type>java.lang.Double</env-entry-type>
            <env-entry-value>245.0</env-entry-value>
        </env-entry>
    </session>
</enterprise-beans>
</ejb-jar>
```

In this example the converter bean can be customized at deployment time with the conversion rate. Dependency injection will take place due to the `@Resource` annotation on the `rate` field of the `ConverterBean` class. The injected value is specified in `ejb-jar.xml` using the `<env-entry>` tag. The component referring to its environment entry is a session bean with the name `ConverterEJB`, as specified with the `name` element in the `Stateless` annotation on the one hand, and with the `<ejb-name>` tag in the deployment descriptor on the other. The name of the environment entry is composed of the fully qualified name of the bean class, and the name of the field. Similarly to `@EJB`, one can use setter methods of JavaBean properties as the target of injected environment entries instead of instance fields (e.g. `setRate`). In such a case the name of the *property* (and not the name of the setter method) is used in the name of the environment entry.

Session context

- Accessing the container from the component
- `SessionContext` interface
  - Lookup in JNDI
  - Other services later...
- Can also be injected

```
import javax.ejb.*;
import javax.annotation.Resource;

@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {
```

```
    @Resource public void setSessionContext( SessionContext ctx ){
       ...
    }
    ...
}
```

A session bean can request information or access certain services from its container through a special object, the session context. A possible use of the `@Resource` annotation is to inject the session context into the session bean. Here we have chosen to annotate a setter method with `@Resource`. In this setter method we could (but do not have to) save the injected `SessionContext` value.

Looking up environment entries in session context

```
import javax.ejb.*;
import javax.annotation.Resource;

@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {

    @Resource public void setSessionContext( SessionContext ctx ){
       rate = (Double)ctx.lookup("rate");
    }

    private double rate;

    public double forintToEuro( double amount ){
       return amount / rate;
    }
}
```

Here we can see an example where the setter method does not save the injected value, it merely uses this value locally: in this case the session context is used to perform a lookup in the environment of the component. This is one of the services provided by the session context. One can look up environment entries in a similar way one would do with a JNDI context.

Setting the value of the environment entry

```
<?xml version="1.0" encoding="UTF-8"?>
<ejb-jar [...]>
<enterprise-beans>
    <session>
```

```
        <ejb-name>ConverterEJB</ejb-name>
        <env-entry>
            <env-entry-name>rate</env-entry-name>
            <env-entry-type>java.lang.Double</env-entry-type>
            <env-entry-value>245.0</env-entry-value>
        </env-entry>
    </session>
</enterprise-beans>
</ejb-jar>
```

The name of the environment entry is simply `rate`. This was used in the lookup call as well. In fact, the lookup was performed within the JNDI context of the component: it was an abbreviation of `java:comp/env/rate`.

Looking up environment entries with JNDI

- Same job with JNDI lookup
- Same as using `ctx.lookup("rate")`
- Same deployment descriptor

Fragment from `ConverterBean`

```
javax.naming.Context ctx = new javax.naming.InitialContext();
try { rate = (Double)ctx.lookup("java:comp/env/rate"); }
finally { ctx.close(); }
```

Fragment from `ejb-jar.xml`

```
<env-entry>
    <env-entry-name>rate</env-entry-name>
    <env-entry-type>java.lang.Double</env-entry-type>
    <env-entry-value>245.0</env-entry-value>
</env-entry>
```

We can look up the environment entry in JNDI using the `javax.naming` API as well. The effect is the same, and so is the deployment descriptor. As mentioned before, looking up `rate` and `java:comp/env/rate` is equivalent: the `InitialContext` created for a component is rooted in a context that is accessible for the component as `java:comp/env` (this is the *absolute name* of the context).

How to handle exceptions?

- Long running applications
- Rigorous exception handling policy
- Resource leaking to be avoided

```
try {
   javax.naming.Context ctx = new javax.naming.InitialContext();
   try { rate = (Double)ctx.lookup("java:comp/env/rate"); }
   finally { ctx.close(); }
} catch( javax.naming.NamingException e ){ ... }
```

- May throw checked `NamingException`
  - Constructor
  - `lookup`
  - `close`

Application exception and system exception

- Application Exceptions: in business logic
  - Passed to clients (part of interface)
  - Application may recover
- System Exceptions: in used services
  - Application probably cannot recover
  - Automatic failover by container
  - Clustered EJB

```
try {
   javax.naming.Context ctx = new javax.naming.InitialContext();
   try { rate = (Double)ctx.lookup("java:comp/env/rate"); }
   finally { ctx.close(); }
} catch( javax.naming.NamingException e ){
   throw new javax.ejb.EJBException(e);
}
```

The EJB specification distinguishes two kinds of exceptions. They are to be thrown differently, they are taken care of by the container differently, and the client of the bean receives them differently.

Application exceptions denote exceptional conditions in the business logic. For example, a bean implementing a shopping cart is requested to remove an item from the

cart which is not actually in the cart: the client can be notified of this problem with a programmer defined exception, which is listed in the `throws` clause of the corresponding method in the business interface of the bean. Then the application may try to recover from this problem. Since application exceptions are an integral part of the control flow of the application, the container has nothing to do with them: the exception raised in the bean will be re-thrown in the client as well. The client perceives the exception as if it were thrown by the reference used in the client to access the bean.

System exceptions denote failures in the services upon which the bean was implemented. For example, a bean cannot open a connection to a database, or cannot access the naming service. The application logic need not be prepared for such exceptions. Typically, it is the task of the system administrator to solve the problem, we should not try to recover in the application code. The EJB specification tells us to handle these system exceptions by notifying the container about them. The container will report these exceptions to the clients in a special way, namely wrapped in exceptions of some special types. Moreover, in certain situations the container may try to recover from the problem. In the case of a clustered EJB component the application server may implement some failover mechanism, by redirecting the client's request to another server, when the primary instance of the bean is out of service.

We will come back to this issue later, and learn how the life cycle of a bean, or a transaction is affected by application exceptions and system exceptions.

In the code above, our bean should be prepared to catch `NamingException`, and since it denotes a malfunction in the services of the container, the bean wraps the exception in an `EJBException` instance, and throws it. This latter exception is used to notify the container about the occurrence of a system exception.

Application and system exception types

System exceptions

- `java.rmi.RemoteException` and subclasses thereof
- A `RuntimeException` not specified as *application exception*
  - Checked exceptions can be wrapped in `EJBException`

Application exceptions

- A checked exception other than `RemoteException`
  - Programmer-defined (custom)
  - Predefined (`javax.ejb.CreateException` etc.)
- A `RuntimeException` marked as application exception
  - Annotated with `ApplicationException`
  - Marked with `<application-exception>` in deployment descriptor

Exceptions in the interface

- System exception `java.rmi.RemoteException`
  - May appear in `throws`
  - Checked exception
  - Typical in older code
- Other system exceptions should not be in `throws`
- Application exceptions are good to see in `throws`
  - Checked application exceptions *must* be in `throws`
  - Unchecked application exceptions?

Lifecycle callback methods

- Container manages life-cycle of components
  - Construction
  - Destruction
  - Activation, passivation
- Container may notify components on life-cycle events
- Bean methods can be annotated or marked in deployment descriptor

Fragment of bean class

```
import javax.annotation.PostConstruct;
...
@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {
    @PostConstruct void init(){ ... }
    ...
}
```

One of the key services of containers is that they manage the life-cycle of the components. The container constructs (instances of the) components, destroys them, and there are additional life-cycle activities that we shall cover later. Sometimes a component wants to be notified of these life-cycle events, because it has to perform activities at certain moments: acquire resources when constructed, release resources when destroyed, and so on. The EJB specification introduced the concept of lifecycle callback interceptor methods. These methods will be called by the container automatically, when a certain moment in the life-cycle of the component is reached. The bean developer can write these special methods, and mark them as `PostConstruct`, for instance (either in the deployment descriptor, or using an annotation).

`PostConstruct` lifecycle callback interceptor method

```
import javax.ejb.*; import javax.naming.*;
import javax.annotation.PostConstruct;

@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {
   @PostConstruct void init(){
      try {
         Context ctx = new InitialContext();
         try { rate = (Double)ctx.lookup("java:comp/env/rate"); }
         finally { ctx.close(); }
      } catch( NamingException e ){ throw new EJBException(e); }
   }
   private double rate;
   public double forintToEuro( double amount ){
                        return amount / rate; }
}
```

In this example an annotation is used to mark a method in the bean class as a `PostConstruct` lifecycle callback interceptor method. As the EJB specification says:

> The PostConstruct callback invocations occur before the first business method invocation on the bean instance. This is at a point after which any dependency injection has been performed by the container.

At this moment the container has already initialized the bean instance, so the bean can already access, for example, its JNDI context containing the `rate` environment entry.

Accessing resource managers

- Container helps access pre-configured resource managers
- Component can look them up or have them injected
- For instance JDBC data sources
  - `javax.sql.DataSource`
  - Reference is typically put in context `java:comp/env/jdbc`

Fragment of `ConverterBean`

```
@Resource(name="jdbc/ConverterDB") DataSource ds;
```

Fragment of product-specific deployment descriptor

```
<session>
    <ejb-name>ConverterEJB</ejb-name>
    <resource-ref>
        <res-ref-name>jdbc/ConverterDB</res-ref-name>
        <jndi-name>java:DefaultDS</jndi-name>
    </resource-ref>
</session>
```

Containers provide access to pre-configured resource managers, such as JDBC data sources, to the components. The component often accesses a resource manager via a resource reference, which can be configured at deployment time. In this case our bean can find, as a special environment entry, a reference to a pre-configured JDBC data source in the JNDI context of the bean: `java:comp/env/jdbc/ConverterDB`. The `@Resource` annotation is used here for dependency injection.

Using JDBC in a bean

```
import javax.ejb.*; import java.sql.*; import javax.annotation.*;
@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {
  @Resource(name="jdbc/ConverterDB") javax.sql.DataSource ds;
  @Resource String errorMsg = "HUF->EUR rate is unknown.";
  public double forintToEuro( double amount ){
     String query =
        "SELECT rate FROM Converter WHERE src='HUF' AND dst='EUR'";
     try (
        Connection c = ds.getConnection();
        Statement s = c.createStatement();
        ResultSet rs = s.executeQuery(query)
     ){ if( rs.next() ){ return amount / rs.getDouble("rate"); }
        else { throw new EJBException(errorMsg); }
     } catch( SQLException e ){ throw new EJBException(e); }
  }
}
```

Now the conversion rate is read from a database. To ensure that the conversion `forintToEuro` always uses an up-to-date conversion rate, each invocation of the method will perform a database query. The SQL resources, namely `Connection`, `Statement` and `ResultSet` are closed at the end of the method (or, more precisely, a best-effort attempt is made to close them). This is guaranteed by the *try-with-resources* statement, which has, in this case, a `catch` branch as well. Any `SQLException` arising

from the use of JDBC is considered a system exception, hence it is wrapped in a `javax.ejb.EJBException`, which is then thrown to the container.

Another system exception might come from the absence of the appropriate conversion rate in the `Converter` table. This is observed when the result set after the query is empty. In this case an error message is passed to the thrown `EJBException` instance.

The configuration of the error message is an interesting issue. It looks like an environment entry (injected by the `@Resource` annotation), but it also has an initialization part in its declaration. This construct allows the definition of default values for environment entries. The default value is the one given by the bean developer in the source code, but it can be conveniently overriden, or customized, at deployment time in a deployment descriptor.

```
<?xml version="1.0" encoding="UTF-8"?>
<ejb-jar version="3.0"
         xmlns="http://java.sun.com/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
                  http://java.sun.com/xml/ns/javaee/ejb-jar_3_0.xsd"
>
   <enterprise-beans>
      <session>
         <ejb-name>ConverterEJB</ejb-name>
         <env-entry>
            <env-entry-name>ConverterBean/error</env-entry-name>
            <env-entry-type>java.lang.String</env-entry-type>
            <env-entry-value>Missing HUF/EUR rate!</env-entry-value>
         </env-entry>
      </session>
   </enterprise-beans>
</ejb-jar>
```

When deploying the bean, the table used by the bean has to be created and filled in the used database (in the above example it is the database available at the JNDI name `java:DefaultDS`). This can be achieved with the following SQL commands.

```
CREATE TABLE Converter ( src VARCHAR,
                         dst VARCHAR,
                         rate DOUBLE,
                         CONSTRAINT pk_constraint PRIMARY KEY (src,dst))
INSERT INTO Converter VALUES ('HUF','EUR',237.8)
```

Using JPA in beans

```
import javax.ejb.*;
import javax.persistence.*;

@Stateless(name="ConverterEJB") @Remote(Converter.class)
public class ConverterBean implements Converter {

    @PersistenceContext private EntityManager em;

    private ConversionPK pk = new ConversionPK();
    { pk.src = "HUF"; pk.dst = "EUR"; }

    public double forintToEuro( double amount ){
        return em.find(Conversion.class,pk).convert(amount);
    }

}
```

In this example another annotation is introduced, `PersistenceContext`. This one is also for dependency injection: for injecting a special kind of dependency, an `EntityManager`.

Here it is assumed that an entity class `Conversion` is available in the persistence unit the used persistence context refers to. This entity class has a composite primary key, described by the primary key class `ConversionPK`. The bean stores an instance of the primary key class, and looks up the corresponding `Conversion` entity with it whenever the `forintToEuro` method is invoked. Using the `Conversion.class` value, which is a class literal, as a type token, `em.find` is able to return a statically well-typed, `Conversion` reference. `Conversion` has a `convert` method, as we will see soon.

An entity class with composite primary key

```
import javax.persistence.*;
import javax.ejb.*;

@Entity @IdClass(ConversionPK.class) @Table(name="CONVERTER")
public class Conversion implements java.io.Serializable {

    @Id String src, dst;
    double rate;

    public double convert( double amount ){ return amount / rate; }
}
```

The entity class presented here uses the `@IdClass` annotation to declare the class for representing the composite primary key of the entity. All constituent attributes of the

entity are annotated with `@Id`. The entity class also defines the convenience method `convert`.

Primary key class for composite key

```
public final class ConversionPK implements java.io.Serializable {
   public String src, dst;
   private static boolean equals( Object left, Object right ){
      return (left == right) || (left != null && left.equals(right));
   }
   @Override public boolean equals( Object other ){
      if( other == this ) return true;
      else if( other instanceof ConversionPK ){
         ConversionPK pk = (ConversionPK) other;
         return equals(src,pk.src) && equals(dst,pk.dst);
      } else return false;
   }
   @Override public int hashCode(){
      return   (src==null ? 0 : src.hashCode())
             + (dst==null ? 0 : dst.hashCode());
   }
}
```

The primary key class is `Serializable`. It repeats the primary key fields of the entity class. Furthermore, it defines equality in terms of `String` content equality, and redefines `hashCode` accordingly, as well. Finally, note that the class has a public no-arg constructor (the default constructor).

Configuring the persistence unit

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0"
             xmlns="http://java.sun.com/xml/ns/persistence">
   <persistence-unit name="converter">
      <provider>org.hibernate.ejb.HibernatePersistence</provider>
      <jta-data-source>java:/DefaultDS</jta-data-source>
      <properties>
         <property name="hibernate.archive.autodetection"
                   value="class"/>
         <property name="hibernate.dialect"
                   value="org.hibernate.dialect.HSQLDialect"/>
      </properties>
   </persistence-unit>
</persistence>
```

You should package the `persistence.xml` file, together with the deployment descriptors, in the `META-INF` directory, when you assemble the EJB component.

Remote and local interfaces

EJBs are primarily used for distributed server side

- EJBs are typically accessible from other VMs, from other computers
- Remote procedure call: RMI over IIOP
- `@Remote` expresses *remote business interface*
- Overhead: network communication, marshalling, unmarshalling

Reduce overhead by local communication?

- Within a JVM
- `@Local` expresses *local business interface*

Comparing remote and local interfaces

- Scalability versus overhead
- Different semantics
- Often different design

So far we have seen examples where EJBs were accessed by remote method invocation. Apparently, he have started to use EJBs exactly for this reason: providing remote access for clients. We have learnt already that the communication behind EJBs is based on the Internet Inter-ORB Protocol, the protocol used among CORBA implementations. Whenever a client calls an EJB component, we are aware of the costs of the call: there will be the cost of network communication, and the overhead of marshalling and unmarshalling data. This is significant. We design the application in such a way that communication between components should not occur too frequently.

Due to the high costs of remote communication, sometimes we would rather choose local communication between components. Local communication means that the communicating partners reside in the same Java Virtual Machine, and hence they can communicate without RPC. For efficiency reasons an EJB component might expose a local business interface, which is similar to a remote business interface, but is intended to be used from the same JVM. Note that this local interface takes away the most important feature, and the strongest advantage, of EJB technology: the scalable, seamless remote communication.

However, in certain scenarios the software architect may decide to couple components, requiring that they are deployed to the same JVM, and this way reducing communication costs. This is typical when the components tend to communicate a lot, and scalability does not seem to compensate the direct performance loss incurred by remoting. Performance is very hard to deal with. Decisions that improve efficiency of communication might prevent scalability, and hence in the long run might become harmful. Always be aware of that when you decide to use local interface to connect components! If you are a bit unsure what to do, you'd better use remote interface.

One might think that using the same business interface for remote and local access is good design, since it allows us to postpone the decision whether remote or local interface should be used. However, this is rarely the case. An obvious semantical difference between remote and local methods is the way parameter passing is implemented. It is well-known that Java uses pass-by-value parameters; for primitive types it means what is expected, but for reference types this means passing references by value. That is a local method call may change an object the reference to which was passed as an argument. In contrary, a remote method call is purely pass-by-value, that is in the case of references not the reference will be passed by value, but the referenced object (or, transitively, objects) will be reconstructed at the callee.

```
public interface Sorter {
    void sort( int[] array );
}
```

This code is completely reasonable for a local interface, but makes no sense for remoting. A remote call would not be able to change the order of the elements in the array passed as actual parameter. In a remote call a copy of the original array would be created in the callee. Sorting the copy would have no effect on the original array.

Changing the interfce into functional style would make it appropriate both for local and for remote use.

```
public interface Sorter {
    int[] sort( int[] array );
}
```

Can you still come up with a situation where the object passed as an argument to a remote method call is changed by the call?

One more thing to emphasize with respect to remote and local interfaces is that in many cases these interfaces are designed differently, even if the semantic differences of parameter passing do not matter. Why should we design remote and local interfaces differently? Local business interfaces are like ordinary, POJO interfaces, but remote business interfaces try to avoid overly fine-grained communication. The right approach to follow when a remote interface is designed is to reduce the number of necessary calls to implement a communication session. In a coarse-grained communication approach you try to combine several small steps into a big one; you prefer

communicating less frequently, and maybe passing more data as parameter to a call. In a local interface each method does a small thing with not so much parameters. In a remote interface a method may do more complex activities, using larger or more complex parameters. Why is is so? Because this is a way to reduce the overhead of remoting. The overhead is proportional to the number of method calls, and it is less sensitive to the amount of data communicated.

EJB with local business interface

```
import javax.ejb.*;
import javax.persistence.*;

@Stateless(name="ConverterEJB") @Local(Converter.class)
public class ConverterBean implements Converter {

    @PersistenceContext private EntityManager em;

    private ConversionPK pk = new ConversionPK();
    { pk.src = "HUF"; pk.dst = "EUR"; }

    public double forintToEuro( double amount ){
        return em.find(Conversion.class,pk).convert(amount);
    }

}
```

Packaging bean with local interface with its client

```
kto@sinuhe ~/converter $ unzip -l ConverterEJB.jar
Archive:  ConverterEJB.jar
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2013-11-22 22:22   META-INF/
       68  2013-11-22 22:22   META-INF/MANIFEST.MF
      600  2013-11-22 22:22   Conversion.class
      770  2013-11-22 22:22   ConversionPK.class
      861  2013-11-22 22:22   ConverterBean.class
      129  2013-11-22 22:22   Converter.class
      759  2013-11-22 22:22   MainBean.class
      146  2013-11-22 22:22   Main.class
      162  2013-11-22 22:22   META-INF/jboss.xml
      488  2013-11-22 22:22   META-INF/persistence.xml
---------                     -------
     3983                     10 files
```

When packaging beans that communicate via local business interfaces, you have to put them into the same archive (either in the same ejb jar, or in the same enterprise archive). It is allowed to package more than one bean into the same ejb jar, like in the above `ConverterEJB.jar`.

Summary

So far...

- coding EJB
- coding clients
- lookup and communication
- packaging and deploying

Still to do

- kinds of EJBs
- operational differences
- operation of container

So far we have learnt much about how to write the code for an EJB, how to develop a client, how to specify details of lookup and communication, how to package components, and how to deploy them. Now it is time to understand how EJBs work, what kinds of Enterprise JavaBeans exist, how they differ, and how they are managed by the container.

Session

Kinds of Enterprise JavaBeans

- Session bean
  - **Stateless**
  - Stateful
  - Singleton
- Message-driven bean
- Entity bean (only in EJB 2.1)

The first example has shown a stateless session bean. We have also seen many variations, extensions, additions to the same example. Our next target is stateful session beans.

What is a session bean?

- Acts on behalf of a client
- Extension of the client on the server side
  - Privileges
  - Session
  - Transaction
- Bean instance

Session beans offer services accessible as (either remote or local) method calls. Clients request these services: often not only a single method call, but several ones in a sequence, that is in a *session*. A session bean method performs activities on behalf of the client. We can regard the application as a set of logical processes, where each logical process corresponds to a client, and a session bean implements some part of the process, the one executed on the server side. In this sense the session bean is the server-side part of its client, or more precisely, the extension of the client on the server side. For this reason some properties of the client should be transfered to the session bean as well.

The session bean should be provided the rights to access data, or perform activities according to the privileges of the client. This is a security point of view. When the session bean is serving a client's request, it should work in the *security context* of the client.

Another important concept is the session. As explained, often more than one subsequent method calls make up a *session*, that is a complete conversation between client side and server side. From the point of view of session beans, it is best to think of clients as processes that perform a session by calling the methods of the bean in some order, according some logic programmed in the client code. In this sense the client starts when the session starts, and the client is over when the session is over. This is how the name ``session bean'' becomes sensible.

During the session, the bean might access transactional resources, such as databases. Transactions are organized within sessions. Sometimes a session should be executed as a single transaction. Sometimes a session is a sequence of transactions. And sometimes certain activities performed within the session need not, or should not, be part of a transaction. But transactions also originate from the client-bean relation. The activities of the bean will be performed in a transactional context of its client.

It is a responsibility of the container to keep track of the client-bean relation, and establish the security and transactional context for the bean.

According to the above discussion, it is appropriate to introduce the terminology of *bean instance*. A bean instance is responsible to serve a client, by executing the methods of the client in a session. However, the concept of bean instance will be a bit fuzzy, once we take into account the performance increasing tricks of the EJB container.

Stateless session bean: the client's perspective

- ``Create'' bean by acquiring a reference to it
- Call business methods
- Calls are independent

The clients' perspective of a stateless session bean is that a session is started by ``creating'' a bean - in fact, clients do not create a bean, but merely acquire a reference to it, either with dependency injection, or by looking up the reference in a naming service (e.g. with JNDI), or through a *SessionContext* (if the client is a bean as well).

Life-cycle of bean instance

- Instances are stored in a pool (idle state)
- When client calls business method:
  - An instance is chosen from pool
  - Serves method call
  - Put back in the pool (idle state)
- More instances, each single threaded
- Calls from same client served by different instances
- No session-long association of client to a bean instance
- Independent calls, no conversational state
- State of bean instance is client-independent
- A system exception destroys the bean instance

A stateless session bean is associated to a client throughout the session in the client's perspective only. The container is free to serve each call from the same session (from the same client) by a different bean instance. The instances, therefore, are stored in an idle state in a pool. When a client calls a business method, an arbitrary instance from the pool is selected, and this instance executes the requested method. The only reason to have multiple instances is that session beans are guaranteed to be single threaded, so concurrent calls from (different) clients can be served concurrently by different instances.

As a consequence, a small number of stateless session bean instances can serve many clients: the number of the bean instances only bounds the number of concurrently served business method calls. The difference is significant if clients perform other activities between business method calls.

A final note must be made about system exceptions: such an exception might leave the bean instance in an inconsistent state, therefore the container will destroy the instance (and may replace it with a fresh instance to keep the pool size fixed). Furthermore, the pool size can be altered by the container dynamically, to accommodate to changes in load.

Stateful session beans

Stateful Session Bean

- `@Stateful` annotation or `<session-type>Stateful</session-type>`
- Multiple method calls in a session
- State maintained during session
  - Stored in instance fields of bean class
  - Session progresses
  - Method parameters may add information to state
- Single threaded

In contrast to the previous example, we will now consider stateful session beans, introduced by `@Stateful` annotation or by specifying the session type as *Stateful* in the deployment descriptor. In these beans a session typically consists of multiple business method calls. However, what makes them really different to stateless session beans, is that some state is maintained during the session. The bean class may define instance fields that the business methods may read and write, and which preserve state between calls of the business methods. As the session progresses, more and more information may be gathered in the state, for instance the client may pass information through the parameters of the called business methods. The collected information is used by the bean instance to implement the business logic of the service.

It is reassuring to know that the container is responsible for serializing executions of business methods within a session. It is guaranteed that the stateful session bean instance will be used in a single-threaded way. No concurrency issues arise, the bean developer does not need to program the bean class in a thread-safe manner.

State

Object state

- Instance of the bean class may have state (instance fields)
- Even stateless session beans
- May be unrelated to client

Conversational state

- State containing information from client
- Maintained by stateful session bean

To implement EJB components, the EJB container maintains instances of the bean class, which is developed by the Bean Provider. The instances of the bean class are objects, and thus they may have instance variables, in which they store the object state. This is not to be confused with the more abstract *conversational state* that stateful session beans have. Even stateless session beans can have object state, where they can store data needed for the implementation of their functionality - but no client-specific information. For example, references to other beans, or resources are often held in such instance variables.

The conversational state is specific to stateful session beans. When using a stateful session bean, the client is guaranteed that the conversational state is preserved between calls of the business methods. Such guarantees on the state (the object state) of stateless beans do not exist. It is useless to store client data in the instance fields of a stateless session bean, because they may change their values between business method invocations.

Statefull session bean: the client's perspective

- ``Create'' bean by acquiring a reference to it
- Call business methods
- Calls are **not** independent (conversational state preserved)
- Terminate session by *removing* the bean

Stateful example: a shopping cart

Business interface

```
public interface Cart {
    void put( String item ) throws CartException;
    boolean contains( String item );
    boolean remove( String item );
    java.util.List<String> contents();
    void order();
}
```

Application exception

```
public class CartException extends Exception {}
```

The idea behind this shopping cart example is that the client acquires a bean to represent a shopping cart, maintaining the contents of the cart (i.e. the conversational state) during the session of buying items in a web-shop. When the client obtains a

reference to a `Cart`, it is expected to be empty. The client uses the `put`, `contains`, `remove` and `contents` methods during shopping, where `put` and `remove` will change the conversational state, while `contains` and `contents` will merely query it. The end of the session is indicated by a call to the `order` method, after which the cart will not be allowed to use any more.

Note that in this example an application exception is defined, as a direct subclass of `java.lang.Exception`. This exception can be thrown by `put`, when an invalid item name is passed as an argument.

Stateful example: bean class

```
import javax.ejb.*;
import java.util.*;
@Stateful(name="CartEJB")   @Remote(Cart.class)
public class CartBean extends LinkedList<String> implements Cart {
    public void put( String item ) throws CartException {
        if( item == null || item.trim().equals("") )
            throw new CartException();
        add(item);
    }
    public boolean contains( String item ){
            return contains((Object)item); }
    public boolean remove( String item ){
            return remove((Object)item); }
    public List<String> contents(){ return new ArrayList(this); }
    @Remove public void order(){ ... clear(); }
}
```

The bean class defines a stateful session bean, using the `@Stateful` annotation. Similarly to the `@Stateless` annotation, the `name` and the `mappedName` annotation elements can be filled here.

The `@Remote` annotation specifies that this bean can be accessed remotely through the `Cart` business interface, which the bean class implements as well.

The implementation of the bean class is based on `java.lang.LinkedList`. For simplicity, we have decided to inherit from this collection class, and indeed, the implementation of the business methods are quite simple in this way. After some sanity checks, `put` adds the item into the cart. If the pre-condition is violated, `CartException` is thrown. Since this is an application exception, it will be delivered to the client as it is.

The other business methods are quite straightforward, though `order` clearly only has a placeholder implementation here. Payment and product delivery should be connected to the shopping cart at this point…

Last, but not least, observe the use of the `@javax.ejb.Remove` annotation on the `order` method, which is intended to terminate the session. This annotation tells the container that after the invocation of this method by the client, the stateful session bean must be removed. The client should not try to use the bean after this session terminating operation.

A remark on the implementation

The definition of `contains` and `remove` is tricky, and heavily dependent on the little bit strange way generic collections are defined since JDK 5.

```
package java.util;
public interface Collection<E> extends Iterable<E> {
    boolean contains(Object o);
    boolean remove(Object o)
    ...
}
```

The declared type of the parameter in this method is `java.lang.Object`, even in `Collection<String>`. This allows objects of any type to be used as actual argument to these methods. Besides that this is more compatible with `java.lang.Collection` operations from the pre-generic era, it is completely safe: if we try to find an item of some incompatible type in the collection, we will simply not find it. For example, one can safely try to find an `Integer` in a `Collection<String>`, in a well-typed program `contains` will return `false`. Similar holds for `remove`.

Due to the `Object` static type of the formal parameter in `Collection`, the methods `contains` and `remove` overload the ones in `List`, and not override them. Casting `item` in the body of the methods of `CartBean` ensures that the `Cart` methods call the corresponding `List` method.

Stateful example: the client

```
import javax.ejb.*;
import javax.annotation.Resource;
@Stateless(name="MainEJB") @Remote
public class MainBean implements Main {
    @Resource private SessionContext ctx;
    public Object main( String[] args ){
        ...
        Cart cart = (Cart) ctx.lookup("ejb/cart");
        ...
        cart.put(...);  ...  cart.remove(...);  ...
        cart.contains(...);   ...  cart.contents();
        ...
```

```
        cart.order();
        ...
    }
}
```

The client of our stateful session bean is a stateless session bean in this example. It acquires a reference to `Cart` through its `SessionContext`, injected by the EJB container. The stateful session starts when the JNDI lookup returns the `Cart` reference to `MainBean`, and the EJB container assigns a `CartBean` instance to the session. The session is over when the `order` method is called on the reference. But in the meantime the `MainBean` can perform calls on the other business methods, as part of the session. For example, `MainBean` can process `args`, as shown in the code below. Note that the `CartBean` should not be used after `order`.

```
import javax.ejb.*;
import javax.annotation.Resource;

@Stateless(name="MainEJB")
@Remote
public class MainBean implements Main {
   @Resource private SessionContext ctx;
   public Object main( String[] args ){
      StringBuilder result = new StringBuilder();
      try {
         Cart cart = (Cart) ctx.lookup("ejb/cart");
         for( String command: args ){
            if( command.startsWith("put ") ){
               try {
                  cart.put(command.substring(4));
                  result.append("Put succeeded.\n");
               } catch (CartException e){
                  result.append("Put failed.\n");
               }
            } else if( command.startsWith("remove ") ){
               if( cart.remove(command.substring(7)) ){
                  result.append("Item removed.\n");
               } else {
                  result.append("Nothing removed.\n");
               }
            } else if( command.startsWith("contains ") ){
               result.append( cart.contains(command.substring(9))
                              ? "It is contained.\n"
                              : "It is not contained.\n"
                            );
            } else if( command.equals("contents") ){
```

```
                result.append(cart.contents()); result.append("\n");
            } else {
                result.append("Unknown command, ignoring...\n");
            }
        }
        cart.order();
        return result.toString();
    } catch( Throwable t ){ return t; }
  }
}
```

It would be a bad idea to inject a `Cart` reference into an instance field of `MainBean` instead of creating the session by a JNDI `lookup`. In that case the stateful session would start when the `MainBean` instance were created, and would not restart after the first call to the `order` method. Hence, if a client invoked the `main` method (the business method) of `MainBean` multiple times, the first invocation would close the session, and subsequent invocations would find the stateful session bean in an illegal state. We need a new session whenever the `main` method is called, and dependency injection into an instance field of `MainBean` would result in a single session.

Life-cycle of stateful session bean instance

- Client acquires reference: a bean instance is associated to it
- Client calls business method
  - Same bean instance serves the call
  - State of the instance is preserved
  - Conversational state
- Session is terminated by removing bean
- A system exception or a timeout destroys the bean instance

It must be clear by now how the life-cycle of stateful session beans differs from that of stateless beans. In the stateful case a bean instance is associated to each client throughout the session. This results in the preservation of the instance variables of the bean between subsequent calls. The session is terminated by the client invoking a method annotated by `@Remove` (or declared as a *remove method* in the deployment descriptor).

The bean instance is destroyed when a system exception is thrown, or when the session times out. Timeouts can be configured in the container to tolerate lost client connections or to sustain abandoned sessions.

This was only a part of the story, however.

Passivation and activation

- Increase the number of concurrent sessions
- Bean instance can be reassigned
  - Session data (client 1) is saved (passivation)
  - Session date (client 2) is loaded (activation)
- Many sessions with smaller amount of bean instances
- Conversational state should be serializable
- Not during transaction

Stateless session beans allow higher throughput than stateful ones, because they can serve more clients at the same time (only the number of concurrent business method executions is bounded by the size of the instance pool). To improve throughput of stateful session beans, containers may implement the activation/passivation mechanism. When a client requests a bean, but the instance pool is empty, the container may serve the client with a bean instance that is serving another client already. How? The state of the bean instance is saved (it is *passivated*, e.g. by object serialization), so that the session can be later restored. The client which has been using this bean instance does not notice that the bean instance is taken away. Its view of the bean remains intact. Next time this passivated client calls a business method, a bean instance is selected by the container, and the saved state is reloaded into that bean instance (*activation*). From the client's perspective a stateful session is served with the same bean, but in reality the subsequent business method calls of the same session may be served by the container with different bean instances.

During passivation the conversational state of the bean is saved. The EJB specification requires this state to be serializable, so that containers can use object serialization for implementing passivation. If references to non-serializable objects are to be stored in the bean, it is a good idea to make those references *transient*. During passivation/activation these references may become *null*, so the bean code should be prepared to recover these objects.

The activation/passivation trick, this throughput optimization, is only allowed when the bean is not in a transaction.

Persistence in stateful bean

```
import javax.persistence.*;
@Entity public class Item implements java.io.Serializable {
    private String name = "";  // not null
    private int quantity = 0;  // not negative
    @Id public String getName(){ return name; }
    public void setName( String name ){
        if( name != null ) this.name = name;
```

```
        else throw new IllegalArgumentException("Null!");
    }
    public int getQuantity(){ return quantity; }
    public void setQuantity( int quantity ){
        if( quantity >= 0 ) this.quantity = quantity;
        else throw new IllegalArgumentException("Negative!");
    }
}
```

In most of the cases some persistence state should also be held in an enterprise application, and a session bean must operate on this persistent state. Imagine a database storing the items available in the web-shop. Using JPA, we can lift the database to the object level by defining an entity class. Assume that you have the `Item` entity with a non-null `String name` (the identifier of the entity) and a non-negative `int quantity`. Property-based access of persistent attributes is specified here.

If we would like to pass values of type `Item` as detached object through a remote business interface, we should make the type `Serializable`.

Improving the session interface

```
public interface Cart {
    java.util.List<String> stock();
    boolean available( String item );
    void put( String item ) throws CartException;
    boolean contains( String item );
    boolean remove( String item );
    java.util.List<String> contents();
    java.util.Map<String,Integer> order();
}
```

In order to learn something new about JPA, we now extend the interface of our stateful session bean with two more methods, `stock` and `available`, which can be used to query the database. Method `stock` lists available items, and method `available` tells whether a given item is in stock. We can also change `order` to return the contents of the cart.

Stateful session bean class with persistence context

```
import javax.ejb.*;
import java.util.*;
import javax.annotation.Resource;
import javax.persistence.*;
```

```
@Stateful(name="CartEJB") @Remote(Cart.class)
public class CartBean extends LinkedList<String> implements Cart {
    @PersistenceContext(unitName="stock") private EntityManager em;
    public List<String> stock(){ ... }
    public boolean available( String item ){ ... }
    public void put( String item ) throws CartException { ... }
    public boolean contains( String item ){ ... }
    public boolean remove( String item ){ ... }
    public List<String> contents() { ... }
    @Remove public Map<String,Integer> order() { ... }
}
```

Dependency on a persistence context will be injected to the stateful session bean. The implementation of some methods remains unchanged.

```
import ...

@Stateful(name="CartEJB") @Remote(Cart.class)
public class CartBean extends LinkedList<String> implements Cart {

    ...

    public boolean contains( String item ){
        return contains((Object)item);
    }
    public boolean remove( String item ){
        return remove((Object)item);
    }
    public List<String> contents(){
        return new ArrayList<String>(this);
    }
}
```

Simplistic implementation of `put`

```
import ...

@Stateful(name="CartEJB") @Remote(Cart.class)
public class CartBean extends LinkedList<String> implements Cart {

    ...

    public boolean available( String item ){ ... }
```

```
    public void put( String item ) throws CartException {
        if( available(item) ) add(item);
        else throw new CartException();
    }
}
```

It makes sense to change the contract of `put`: it should throw the application exception `CartException` when the `item` passed as argument is not `available`.

For the sake of simplicity, we do not modify the stock database when an item is put into the cart. Decreasing the stock will only happen in `order`. This is an *optimistic* approach: if there is at least one instance of a given item in stock, we allow the `put` method to put one in the cart. This might result in overbooking, which will be handled in method `order`, as we will see.

Now we can focus on the new methods, and on the new implementation for `order`.

Queries

- Java Persistence query language (JPQL)
- String-based dynamic query

```
@PersistenceContext(unitName="stock") private EntityManager em;

@SuppressWarnings("unchecked")
public List<String> stock(){
    String q = "SELECT i.name FROM Item AS i WHERE i.quantity > 0";
    return em.createQuery(q)
             .getResultList();
}
```

In the implementation of `stock` a query expressed in the Java Persistence query language appears. JPQL allows us to use *string-based dynamic queries*, as well as static queries expressed with meta-data. Here we can see the former. The entity manager API provides a `createQuery` method, expecting the query as a `String` (we use a local variable here to let the example fit on the page). The method `getResultList` returns all query results in a *raw type* `List`. The `@SuppressWarnings` annotation applied here tells the compiler not to emit warning when the value of the raw type is converted to `List<String>` in the `return` statement.

The Query Language resembles SQL pretty much. However, it also exposes object-oriented features.

Queries with parameters

```
PersistenceContext(unitName="stock") private EntityManager em;

public boolean available( String item ){
    String q = "SELECT i.name FROM Item AS i " +
               "WHERE i.name = :name AND i.quantity > 0";
    List list = em.createQuery(q)
                  .setParameter("name",item)
                  .getResultList();
    return !list.isEmpty();
}
```

In the `available` method again a JPQL dynamic query is applied, but this time the query has a named parameter as well. The name of the parameter is `name`. Syntactically, these parameters appear in the query preceded with a colon, and the value is set with the `setParameter` (family of) method(s).

Sequencing operations on `Query`

```
List list = em.createQuery(q)
              .setParameter("name",item)
              .setMaxResults(1)
              .getResultList();
}
```

`Query` API documentation

```
Query setMaxResults(int maxResult)

Set the maximum number of results to retrieve.
Parameters: maxResult - maximum number of results to retrieve
Returns: the same query instance
Throws: IllegalArgumentException - if the argument is negative
```

The `Query` interface uses a nice trick to make the API easier to use. Methods, declared with `void` return type in similar situation elsewhere, return `this` in this interface (as required by the API documentation). This way these methods can be sequenced, like in the code above. For example, consider the `setMaxResults` method of the `Query` interface, which can be used to specify an upper bound on the number of returned query results. We can mix in the call to this method in the above way.

Aggregate expression

```
public boolean available( String item ){
    String q = "SELECT COUNT(i) FROM Item AS i " +
               "WHERE i.name = :name AND i.quantity > 0";
    Object result = em.createQuery(q)
                      .setParameter("name",item)
                      .getSingleResult();
    return (Integer)result > 0;
}
```

Obviously, this query can also be written with an aggregate expression, i.e. `COUNT`.

The above query returns exactly one result, therefore the `getSingleMethod` method of `Query` can be used here, which returns an untyped query result as an `Object` reference. (A values of a primitive type is converted to the respective wrapper class.)

Closing the session

```
@Remove public Map<String,Integer> order() {
    Map<String,Integer> contents = new HashMap<String,Integer>();
    for( String name: this )
        if( contents.containsKey(name) ){
            contents.put(name,contents.get(name)+1);
        } else { contents.put(name,1); }
    for( String name: contents.keySet() ){
        Item item = em.find(Item.class,name);
        int quantity = item.getQuantity();
        int requested = contents.get(name);
        int fulfilled = Math.min( requested, quantity );
        item.setQuantity(quantity-fulfilled);
        contents.put(name,fulfilled);
    }
    clear();
    return contents;
}
```

The implementation of `order` is long, but straightforward. First we iterate through the contents of the cart (`List<String> this`), and collect the multiplicity of items which occur in the cart in a map. Then we go through this map, and check how many instances of the given items are in stock. In the case of overbooking, we only allow `order` to buy no more instances of a given item than available (`fulfilled`). The return value of `order` describes how many instances of each item were actually bought, decreasing the stock by these amounts. Finally, the cart is cleared.

Transactions

- Examples used transactions
- Implicit, based on defaults
  - Container managed transactions
  - Java Transaction API based transactions
  - *Required* transaction attribute
- Separate chapter...

Although transactions have not been used explicitly, all our examples so far were transactional. Transactions are essential in EJB, therefore a complete chapter is devoted to this topic. It is helpful to study that chapter before continuing this example.

In the preceding examples transactions were managed by the container, the programmer did not have to write any code for transaction management. In these cases, the container relies upon the Java Transaction API to set transaction boundaries. The default mechanism is to run all bean methods in some transaction, but only start transaction when necessary (when there is no active transaction already). This is expressed with the *Required* transaction attribute.

Verifying transaction boundaries

```
import ...

public @Stateful(name="CartEJB") @Remote(Cart.class)
class CartBean extends LinkedList<String>
            implements Cart, SessionSynchronization {

    public void put( String item ) throws CartException { ... }
    public boolean remove( String item ){ ... }
    ...
    @Remove public Map<String,Integer> order() { ... }

    public void afterBegin(){ ... }
    public void beforeCompletion(){ ... }
    public void afterCompletion(boolean commit){ ... }
}
```

The `javax.ejb.SessionSynchronization` interface gives us a way to get notification from the container about transaction life-cycle events. If a bean class implements this interface and the three methods within this interface, the container will call these methods when a transaction is started by the bean, and when a transaction is completed by the bean. One method is called just *before* the transaction completes, and

another one just *after* the transaction completes. The latter one is given a boolean argument specifying whether the transaction completion is a commit or a rollback.

These three callback methods can be used for different purposes - here we just want to log when transactions are started and completed. Let's see how.

Logging with Log4J

```
import ...
@Stateful ... class CartBean ... implements SessionSynchronization
    ...
    private org.apache.log4j.Logger logger;
    public void afterBegin(){ logger.info("Tx is started."); }
    public void beforeCompletion(){
        logger.info("Tx is about to complete.");
    }
    public void afterCompletion(boolean commit){
        logger.info("Tx is "+(commit?"committed.":"rolled back."));
    }
    @PostConstruct @PostActivate private void setLogger(){
        logger = logger.getLogger(getClass());
    }
    @PrePassivate private void unsetLogger(){ logger = null; }
}
```

Just for fun, we use Apache Log4J for logging. Info-level log messages are emitted in the `SessionSynchronization` methods using the logger attached to the bean. But how is this `logger` initialized? The `PostConstruct` life-cycle callback method makes sure that this field is initialized. (Quiz: if it is this method that initializes the `logger` field, why does not it throw a `NullPointerException` when invoking the `getLogger` method on the yet uninitialized `logger` reference?)

There is one more lesson to learn here: the role of the `PostActivate` and `PrePassivate` life-cycle callback methods. You can probably figure out that these methods are related to the passivation/activation mechanism, which can be used by the container to improve the throughput of stateful session beans. When a bean is passivated, the state of the bean instance is saved. This can happen by object serialization or by other means. However, it is required that this state be serializable. By unsetting `logger` before passivation, and resetting it after activation, we can guarantee that the logger object is never saved.

Note: while Log4J is the default logging mechanism in JBoss, and hence quite easy to use, it is not as easy at all to get it work in GlassFish. Find on the web how to configure log4j for GlassFish!

Investigating the logs

```
import ...

public @Stateful(name="CartEJB") @Remote(Cart.class)
class CartBean extends LinkedList<String>
            implements Cart, SessionSynchronization {

    public void put( String item ) throws CartException { ... }
    public boolean remove( String item ){ ... }
    ...
    @Remove public Map<String,Integer> order() { ... }

    public void afterBegin(){ logger.info("Tx is started."); }
    public void beforeCompletion(){ ... }
    public void afterCompletion(boolean commit){ ... }
}
```

If we run our bean with our existing client, it turns out that each business method starts a new transaction, and commits it upon termination. This is the default behaviour, the result of the implicit `Required` transaction attribute.

Note: the `order` method might behave a bit differently to the other business methods: since this method removes the bean, the `beforeCompletion` and `afterCompletion` methods might not be called on the bean instance at the end of the transaction.

Logging even more

- Business interceptor
- AOP-style *around*
- Good for many things

Inside `CartBean`

```
@AroundInvoke private Object logging( InvocationContext ctx )
throws Exception {
     logger.info("method starts");
     Object result = ctx.proceed();
     logger.info("method stops");
     return result;
}
```

We can make the beginning and the end of the business methods visible in the logs by adding logging instructions to the business method implementations. This way

we can better observe when the calls to the `SessionSynchronization` methods are performed by the container. To make this additional logging more convenient, we apply a nice trick - which is applicable for many purposes, by the way.

The trick is to use the `AroundInvoke` annotation. With this annotation, business method executions can be surrounded by programmer-specified code. This is very similar to *around advice* known from Aspect-Oriented Programming in general, and the AspectJ language in particular. Whenever the container is about to call a business method on the bean instance, it first calls the `AroundInvoke` method (`logging` in this case), passing information about the business method to be called. The `AroundInvoke` method can call the original business method if, and whenever, it likes. This is achieved by `ctx.proceed()`. When the business method terminates, the `AroundInvoke` is still under execution, so it can perform further activities before returning the control to the container.

Now let's go back to our original problem.

The programmer might want to start a single transaction for the whole session. (Remember, the session starts when the client acquires a reference to the stateful session bean, and the session terminates when the client calls a method annotated with `@Remove`: in this case the `order` method.) In order to achieve this transactional behaviour, we should start and commit the transaction in the client of the session.

Transaction in the client

```
public class Shopping {

   @EJB static Cart cart;
   @Resource static UserTransaction tx;

   public static void main( String[] args ) throws Exception {
       tx.begin();
       ...
       cart.put(...);
       ...
       cart.remove(...);
       ...
       cart.order();
       tx.commit();
   }
}
```

In this scenario, the client runs a single transaction, and all activities performed with the session bean during the client session become part of this transaction.

Forcing the client to start transaction

```
import ...

public @Stateful(name="CartEJB") @Remote(Cart.class)
@TransactionAttribute(TransactionAttributeType.MANDATORY)
class CartBean extends LinkedList<String> implements Cart {

    public void put( String item ) throws CartException { ... }
    public boolean remove( String item ){ ... }
    ...
    @Remove public Map<String,Integer> order() { ... }

}
```

One can use the transaction attribute `MANDATORY` to tell the container that the bean expects its client to start a transaction before calling the bean's methods. In contrast to `REQUIRED`, with `MANDATORY` the container will not start a transaction for the bean method if the caller is not in a transaction, but throw an exception instead.

This can be used to prevent unintended non-transactional calls to the bean methods.

Bean managed transactions

- Stateful session beans
- More flexibility
- Transactions started/completed in different methods
- Transaction can be started/completed in the middle of a method

For stateful session beans one might consider using bean managed transactions instead of container managed transactions. BMT requires a bit more work, but it is more flexible as well. Sometimes it is necessary to start a transaction in one method, and commit it in another one. This cannot be achieved with CMT. Moreover, with BMT the programmer can start a transaction, or commit one, in the middle of the method, or conditionally. Many applications do not need this flexibility - for those CMT is sufficient. But do not hesitate to use BMT if the extra flexibility is advantageous.

BMT example

```
@Stateful @Remote(Cart.class)
@TransactionManagement(TransactionManagementType.BEAN)
public class CartBean extends LinkedList<String> implements Cart {
```

```
    @PersistenceContext(unitName="stock") private EntityManager em;
    @Resource private UserTransaction tx;

    public void put( String item ) throws CartException {
        if( available(item) ){ add(item); }
        else { throw new CartException(); }
    }

    ...
}
```

Note the `TransactionManagement` annotation here. In this code the programmer will demarcate transactions using Java Transaction API, and the `UserTransaction` interface. We must make sure that business methods run in a transaction.

Starting transactions in `CartBean`

```
private boolean inTx = false;

@AroundInvoke private Object ensureTx( InvocationContext ctx )
throws CartException {
    String message = inTx ? "continued" : "begun";
    if( ! inTx ){
        try {
            tx.begin();
            inTx = true;
        } catch( Exception e ){ throw new EJBException(e); };
    }
    try { return ctx.proceed(); }
    catch( CartException e ){ throw e; }
    catch( RuntimeException e ){ cleanUp(); throw e; }
    catch( Exception e ){ assert false; throw new EJBException(e); }
}
```

We are going to use the `AroundInvoke` interceptor method again. Before executing a business method we will check whether a transaction has been started already,and if not, we start one.

Rolling back the transaction in `CartBean`

```
@PreDestroy private void cleanUp(){
    clear();
    try {
```

```
        if( inTx ){
            tx.rollback();
            inTx = false;
        }
    } catch( Exception e ){ throw new EJBException(e); };
}
```

If something goes wrong, i.e. a `RuntimeException` is thrown, we can roll back the transaction by calling this `cleanUp` method. This method is also called by the container when it destroys the bean, for instance when timing out a client. This is achieved by adding the `PreDestroy` annotation to the method.

Committing the transaction in `CartBean`

```
@Remove public Map<String,Integer> order() {
    Map<String,Integer> contents = new HashMap<String,Integer>();
    ...
    try {
        tx.commit();
        inTx = false;
    } catch( Exception e ){ throw new EJBException(e); }
    clear();
    return contents;
}
```

Finally, when the workflow completes normally, i.e. by the client calling the `order` business method, we commit the transaction and clear the bean instance (so that it does not consume resources while it is not assigned to clients).

Singleton session bean

Kinds of Enterprise JavaBeans

- Session bean
  - **Stateless**
  - **Stateful**
  - Singleton
- Message-driven bean
- Entity bean (only in EJB 2.1)

The third kind of session beans is the singleton bean. They were introduced in EJB 3.1, so they are quite new. It turns out that they are quite useful to accomplish tasks that were very hard to perform before EJB 3.1. However, many developers misunderstand the concept behind singleton beans, and attribute properties to them that they simply do not have. Singleton session beans provide a structured way to implement, at least partially, the singleton design pattern in the EJB world.

The singleton design pattern, also considered by many as an anti-pattern, can be quite problematic in a distributed environment. Furthermore, singletons introduce performance bottlenecks in concurrent/parallel applications.

Singleton beans

- `Singleton` (cf. `Stateless` or `Stateful`)
- One per application per JVM
- May hold state
  - Lost when server crashes
- Shared by clients
  - Same bean instance
  - Concurrency-awareness
  - Not destroyed in the case of system exception

We can define a singleton bean with the `@Singleton` annotation or with `<session-type>Singleton</session-type>` in the EJB JAR deployment descriptor. The most important property is that in any application, there is a single instance of a singleton bean per JVM. The EJB specification explains that:

``A singleton session bean is a session bean component that is instantiated once per application. In cases where the container is distributed over many virtual machines, each application will have one bean instance of the singleton for each JVM.''

Many people read only the first sentence (and emphasize ``once per application''), which is made more precise -- or, one can say, is actually annulled -- by the second sentence: in a clustered EJB container there is a bean instance in every JVM. (This behaviour is quite similar to servlets. Indeed, some functionalities provided by singleton beans were programmed with servlets before EJB 3.1.)

A key feature of EJB technology is its support for clustered environments. Clustering is the fundamental way to improve performance (scaling out), as well as to ensure fault-tolerance. It is therefore a big mistake to believe that singleton beans are *really* singletons in our application. An application written with the above assumption in mind might work for a while (until the EJB container is not clustered), but may break abruptly (when deployed in a clustered environment).

Singleton beans may hold state. This state is not a conversational state between a bean (instance) and its client, like with stateful beans. It is not a state containing merely

implementation-specific, auxiliary data either, as in the case of a stateless session bean. The state in a singleton bean is used by all its clients, i.e. for sharing information or even for communication.

Often people say that a singleton bean is instantiated only once. To make this statement more precise, you should say: they are instantiated only once during each execution of the EJB container. If the container crashes, a completely new instance will be created, and the previous state of the bean will be lost. Again, the EJB specification explains that:

``Once instantiated, a singleton session bean instance lives for the duration of the application in which it is created. It maintains its state between client invocations but its state is not required to survive container shutdown or crash."

The next important note is that singleton beans are shared by clients. While stateful and stateless session beans are tied to a single client, at least logically, a singleton bean will be used by multiple clients: this is exactly why we need them. Stateful and stateless session beans can be developed without minding concurrency at all. Singleton beans, however, do have to be aware of concurrency, e.g. synchronizing threads.

Singleton beans react upon system exceptions differently to other session beans: the container does not discard the bean if such an exception is raised.

Managing concurrency

- Container-Managed Concurrency
  - default
  - `@ConcurrencyManagement(ConcurrencyManagementType.CONTAINER)`
  - Tuning: `@Lock`, class or method
    - `@Lock(LockType.READ)`
    - `@Lock(LockType.WRITE)` (default)
- Bean-Managed Concurrency
  - `@ConcurrencyManagement(ConcurrencyManagementType.BEAN)`

There are two ways to manage concurrency in a singleton bean: the do-it-yourself approach, when the programmer uses Java language or API features to provide synchronization, or by asking the help of the container. The latter is more convenient, and might be sufficient in many cases. One can choose between the two approaches with the `ConcurrencyManagement` annotation.

If we choose to use container managed concurrency in a singleton bean, which is the default behaviour, we can attach locking policies to each business method of the bean: the policies implement the reader-writer synchronization pattern, by exposing read and write locks. We can set the locking policy either on the whole class, or on

method-level (which overrides the locking policy on the class level). The default for locking policy is the use of write locks.

Write locks are exclusive locks: once a thread acquires a write lock, no other threads can acquire another lock (until the first one releases the write lock, of course). Read locks, on the other hand, allow concurrency to take place, they only exclude subsequent write locks. Finally, note that reader-writer synchronization may exhibit liveness anomalies, e.g. starvation of writers. The EJB container should be careful to avoid starvation.

Example singleton bean

```
@Singleton
public class DataStoreBean {
    private SomeDataStructure state;

    @Lock(LockType.READ)
    public Item readSomeData( Input parameter ) {
        ...
    }

    @Lock(LockType.WRITE)
    @AccessTimeout(value=360000)
    public void synchronizeWithDatabase() {
        ...
    }
}
```

Assume a singleton bean that holds some state in the memory and makes this state available to clients through the `readSomeData` method. The state is (re-)read from a database when `synchronizeWithDatabase` is called. A user story of this bean may explain that the latter method is invoked rarely (e.g. once per day) by a special client (e.g. by one with administrator privilege), while other clients call the first method concurrently, reading the memory cached data very efficiently.

The latter method is annotated with `AccessTimeout`, which means that a call to this method is cancelled if it cannot complete within the given amount of time.

Reentrance

- When singleton bean calls itself through container
- No write lock with read lock
- `IllegalLoopbackException`
- Deadlock avoided…

- ... but only for single-JVM invocations

A singleton bean can call its own method either directly, or through the container (by acquiring an ejb reference to itself). In the latter case container managed locks raise a question: are locks re-entrant or not? The answer is yes - but with a small remark: if a thread holds a read lock on a singleton bean and attempts to acquire a write lock, the container will deny this and throw `IllegalLoopbackException`. This approach helps avoid deadlocks. However, if more than one containers are involved in such a loopback call, they are not required to detect the above read-then-write conflict, so deadlock might occur in such a case.

Singleton life-cycle

- Lazy or eager instantiation
  - `@Startup`
- Callbacks
  - `@PostConstruct`
  - `@PreDestroy`

```
@Singleton @Startup public class DataStoreBean {
    ...

    @PostConstruct @Lock(LockType.WRITE)
    public void synchronizeWithDatabase() { ...  }
}
```

We have learnt so far that a single instance of a singleton beans may be used in more than one threads concurrently (depending on the implemented synchronization behaviour), and that system exceptions in such a thread will not destroy the instance. One question on the life-cycle is, however, when the instance is created. If the bean is defined with the `Startup` annotation, the container instantiates the bean eagerly, that is at system startup. Without that annotation lazy instantiation takes place.

Singleton beans may require to be notified about life-cycle events, which is achieved by the post-construct and pre-destroy callback methods.

Application startup-shutdown

Another use case for Singleton bean: notification about application life-cycle

- Startup and PostConstruct
- PreDestroy

The post-construct notification together with the eager initialization reveals a nice application facility of singleton beans: one can perform application-level initialization in the `@PostConstruct` method of a `@Startup` singleton bean.

Similarly, since a singleton bean is only destroyed by the container when the system is shut down, application-level shutdown activities can be programmed in the `@PreDestroy` method of a singleton bean.

Instantiation order

Partial order can be specified with `@DependsOn`

Dependencies

```
@Singleton @Startup public class Bean1 {
    @PostConstruct public void init(){ ... }
}

@Singleton @Startup @DependsOn("Bean1")
public class Bean2 {
    @PostConstruct public void init(){ ... }
}
```

Especially when multiple singleton beans are used to initialize the same application with their `@Postconstruct` methods (but also under other circumstances) it might be desirable to specify the instantiation order of beans, and hence the order of the execution of the `@Postconstruct` methods. This can be achieved with the `DependsOn` relation. One can specify a partial order among the beans, which will be respected by the container when the beans are instantiated.

Message-Driven Beans

Synchronous and asynchronous communication

- Session beans
  - Mostly with synchronous method invocations (local or remote)
  - `@Asynchronous` call is also possible
    - `void` or `Future<V>` (`javax.ejb.AsyncResult<V>`)
    - Long-running operation, processor-intensive or background task
    - Increase application throughput, improve application response time
    - Both communication parties must be available
- Message driven beans

Synchronous communication in a distributed system results in stronger coupling of components than in the case of asynchronous communication. Furthermore, synchronous communication often leads to performance degradation. Reducing coupling and the need for waiting is a good idea. Method calls can be made asynchronous in EJB by annotating methods with `Asynchronous`. Such methods should either return `void` (so that clients need not wait for results) or a future (so that clients can get the result of the call later). The second case is a bit trickier: when the client calls the method, the container responds instantaneously, and returns a future. In the meantime the business method in the session bean starts execution. When the client wants to use the result of the computation, it requests the value stored in the future; this is a blocking operation, and requires the completion of the business method in the session bean.

To implement futures, the EJB API offers the `AsyncResult` generic class.

This technique is especially useful when the session bean has long running operations, or it offers methods that can run in the background.

Note, however, that in the case of asynchronous session bean calls both the bean and its client must be available and active at the same time. Even better separation of components can be achieved by supporting offline communication, which is made possible by message-oriented middleware. Message driven beans let us exploit this facility.

Message-driven bean

- Accessible by JMS messages
- Asynchronous receiver

```
import javax.ejb.*;
import javax.jms.*;

@MessageDriven(...)
public class ProcessOrders implements MessageListener {
    public void onMessage(Message msg) {
        try { ObjectMessage omsg = (ObjectMessage)msg;
              Ordered item = (Ordered) omsg.getObject();
              ...
        } catch ( Exception e ){ ... }
    }
}
```

Message driven beans are based on JMS, the Java Message Service specification. Such beans are asynchronous receivers of JMS messages, expressed by implementing the `javax.jms.MessageListener` interface. This interface requires the `onMessage` method, which is called by the container when a new message is delivered by the

JMS-compliant middleware. JMS guarantees a single threaded execution model for message listeners; the EJB container is responsible for calling `onMessage` on a single thread. In this example the message is type cast to `ObjectMessage`, the body of which is a serializable Java object.

Nothe that the bean class is annotated with `javax.ejb.MessageDriven`. This annotation can configure the used JMS destination.

Configuring JMS destination

```
@MessageDriven( activationConfig = {
    @ActivationConfigProperty( propertyName="destination",
                               propertyValue="orders"),
    @ActivationConfigProperty( propertyName="destinationType",
                               propertyValue="queue")
})
public class ProcessOrders implements MessageListener { ... }
```

This is how the `MessageDriven` annotation looks like in our example. Its annotation element, `activationConfig` is assigned a sequence of properties. One of the properties describes that the JMS destination is a queue (rather than a topic), and the other property specifies the name of the used JMS resource.

Sending JMS message

```
@Resource(mappedName="java:JmsXA") QueueConnectionFactory qfactory;
@Resource(mappedName="orders") javax.jms.Queue queue;

private void sendOrder( Ordered item ) {
   try {
      QueueConnection connect = qfactory.createQueueConnection();
      try {
         connect.start();
         QueueSession session = connect.createQueueSession(true,0);
         QueueSender sender = session.createSender(queue);
         ObjectMessage message = session.createObjectMessage(item);
         sender.send(message);
      } finally { connection.close(); }
   } catch( Exception e ){ throw new EJBException(e); }
}
```

The client of a message driven session bean uses the JMS API to send messages to the used destination. If the client is component, (e.g. a session bean) the container can dependency-inject the necessary JMS resources.